

RAPID: Robot Agentic Programming from Demonstrations

Abstract—Coding agents solve complex programming problems through an iterative loop of generation, execution, verification, and revision. Bringing this agentic coding loop to robotic manipulation, however, typically assumes task success signals, human-engineered manipulation primitives, and interactive environments for verification. We introduce Robot Agentic Programming from Demonstrations (RAPID), a framework for generating and refining robot programs from a *single visual human demonstration*. RAPID uses the demonstration to infer the task specification, construct task-specific manipulation primitives, reconstruct an interactive simulation environment, and generate scene variants for verification and refinement. To make the resulting programs reusable beyond the demonstrated scene, we introduce a *relational program representation* that captures the structure of the demonstrated strategy rather than its specific motions. Primitives are expressed as trajectory-optimization programs that realize object-level motion effects, while their composition is defined through relational constraints that resolve scene-dependent geometry at execution time. We primarily evaluate RAPID on eight contact-rich nonprehensile manipulation tasks, where it outperforms baselines under substantial variations in object pose, geometry, material, and scene configuration. To assess generality beyond nonprehensile tasks, we further evaluate RAPID on LIBERO-Pro, which features prehensile manipulation tasks. Finally, we demonstrate reliable deployment on a real robotic system.

I. INTRODUCTION

Recently, coding agents [1], [2] have shown remarkable capabilities in developing complex software systems, sparking growing interest in solving robotic manipulation tasks through programming [3]–[9]. Consider how a human engineer would develop a robot program for the task in Figure 1 before the advent of coding agents: given a *task* (pick up the cereal box), they would implement a program *representing* the solution strategy (push→flip→grasp), and *verify* the program on test cases and iteratively debug it. This workflow mirrors the core strength of coding agents: not one-shot code generation, but the iterative cycle of generating, executing, verifying, and revising a program, commonly referred to as the agentic coding loop [10], [11].

However, bringing this agentic coding loop to robotic manipulation requires several ingredients that are not naturally available: (a) a task specification that defines the intended outcome, (b) manipulation primitives that bridge program-level reasoning and low-level execution, and (c) interactive environments for verification. Existing robotic coding-agent systems typically assume these ingredients are provided a priori, through ground-truth rewards and success signals, human-engineered primitives, and resettable environments. This raises a fundamental question: **how can we close the agentic coding loop without assuming these ingredients being provided a priori?**

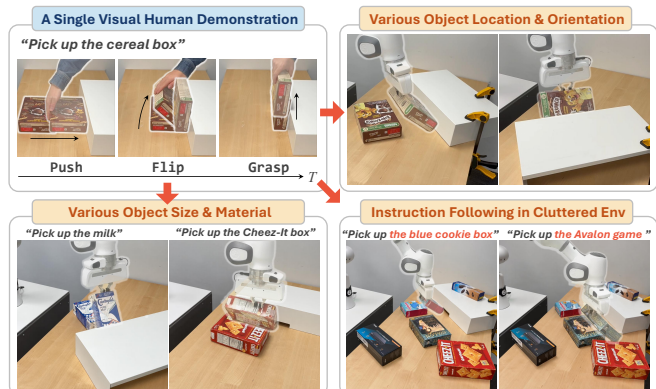


Fig. 1: RAPID (Robot Agentic Programming from Demonstrations) generates a reusable manipulation program from a single visual human demonstration that generalizes across variations in object appearance, pose, geometry, physical properties, and scene configuration.

Our key idea is to use *demonstrations* as the programming interface for supplying the task- and motion-level information needed by the agentic coding loop. A demonstration directly shows a feasible way to accomplish a task and, together with a language description, conveys the intended outcome. Motivated by this perspective, we introduce **Robot Agentic Programming from Demonstrations (RAPID)**, a framework for generating and iteratively refining robot programs from a *single visual human demonstration* with a coding agent. From the demonstration, RAPID infers the task specification, constructs the manipulation primitives and composes them into a strategy. It further reconstructs an interactive simulation environment from the demonstration for program verification and refinement, and generates scene variants to improve generalization. In this way, a single demonstration bootstraps the task specification, manipulation primitives, and verification environment for closing the agentic coding loop.

Rather than merely reproducing the demonstration, we aim to extract a reusable and generalizable manipulation program. Programs that directly encode poses, distances, or waypoints may succeed in the demonstrated scene, but can fail when the scene changes. In this work, we propose a *relational program representation*, where the solution strategy is represented by object-centric relations and effects. Specifically, RAPID writes primitives as local trajectory-optimization programs that realize object-level motion effects, such as pushing an object by a displacement or flipping it by an angle, while resolving scene-dependent geometry at execution time. At the task level, primitives are composed into a strategy through relational constraints that determine how the outcome of one phase specifies the next. For example, in Figure 3, rather than replaying a fixed distance, the program first computes the goal position from the relation between the brown obstacle

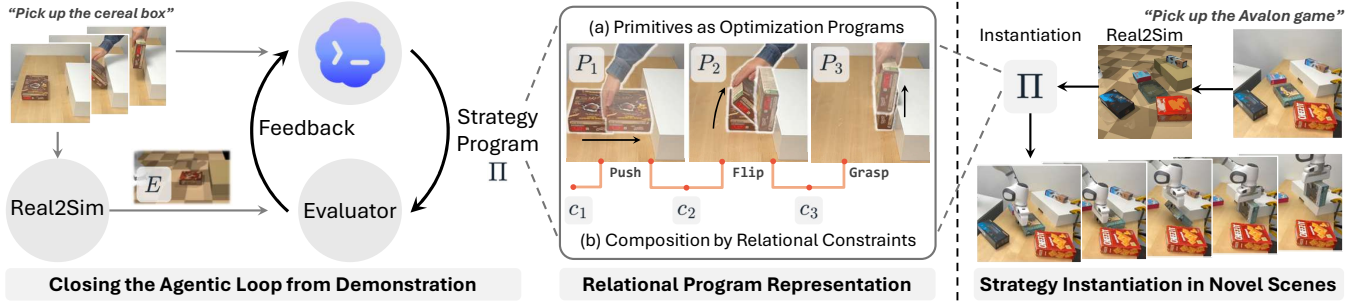


Fig. 2: Overview of RAPID.

and the white fixture, and then determines how far to move the target object during the current primitive (press move) based on the state produced by the preceding primitive (flip). By expressing primitives and their composition relationally, RAPID preserves the invariant structure of the demonstrated strategy while adapting to changes in object appearance, pose, geometry, physical properties, and scene configuration.

The remaining challenge is *verification*: the coding agent needs an interactive environment in which candidate programs can be executed, evaluated, and revised. RAPID reconstructs such an environment in simulation from the demonstration and augments it with scene variants generated by perturbing the demonstrated scene. The coding agent then generates task-specific filters that discard variants that are physically infeasible or inconsistent with the task. Afterwards, the coding agent executes candidate programs across this verification set and iteratively revises them based on observed failures.

In this work, we focus on *nonprehensile manipulation*, a particularly challenging domain in robotic manipulation. Many prehensile tasks can be composed from a compact and well-established set of reusable primitives [12], such as *pick*, *move in free space*, and *place*. Nonprehensile manipulation, by contrast, involves a much broader range of geometry- and contact-dependent interactions, including pushing [13], pivoting [14], toppling [15], flipping [16], and sliding [17]. Capturing this diversity with a fixed primitive library is difficult, making manual primitive design a substantial engineering burden. Nonprehensile manipulation therefore provides a natural testbed for studying whether a coding agent can construct reusable manipulation primitives from a single demonstration rather than relying on human-engineering.

We evaluate RAPID on eight contact-rich nonprehensile manipulation tasks in simulation, asking whether programs constructed from a single demonstrated instance can generalize to substantial variations in object appearance, pose, geometry, physical properties, and scene configuration. RAPID outperforms baselines without relying on pre-specified success signals, human-engineered manipulation primitives, or resettable environments. To evaluate the generality of RAPID beyond nonprehensile manipulation, we additionally test it on LIBERO-Pro [18], a simulation benchmark featuring prehensile manipulation tasks. Finally, we demonstrate that RAPID can be deployed reliably on a real robotic system.

II. PROBLEM STATEMENT

We consider programming a reusable *nonprehensile manipulation* strategy from a single visual human demonstration

D and a language instruction I . The demonstration records a human performing the task in an example scene. The resulting program should solve a family of task variants $\mathcal{E}$ that share the same task objective but may differ in object appearance, geometry, pose, physical properties, and scene configuration.

We assume that no manipulation primitives, explicit success criterion, or instantiated simulation environment are provided a priori. Given only (D, I) , the agent must first construct a set of executable manipulation primitives

$$\mathcal{P} = \{P_1, \dots, P_N\},$$

and then compose these primitives into a strategy Π . The resulting program should capture the manipulation structure underlying the demonstration and generalize beyond the demonstrated instance:

$$(D, I) \longrightarrow (\mathcal{P}, \Pi), \quad \Pi(E; \mathcal{P}) \text{ succeeds for } E \in \mathcal{E}.$$

The goal is to recover reusable primitives and a strategy that can adapt to unseen task variants.

III. RELATIONAL MANIPULATION PROGRAMS

As shown in Figure 2, RAPID uses agentic programming to construct a reusable robot manipulation strategy Π given a single visual human demonstration D and its corresponding language instruction I . It first infers the task specification from (D, I) and generates an initial object-centric, relational program for accomplishing the task. RAPID then reconstructs an interactive simulation environment from D , in which it iteratively executes, verifies, and revises the program. To improve reusability and generalization, RAPID also proposes several scene variants, verifies the program across them, and uses the failures to guide revision. At deployment, the strategy is frozen and can be instantiated in a novel scene through lightweight semantic binding.

In this section, we introduce the object-centric and relational program representation of primitives, how primitives are composed to form a strategy, and how a strategy is instantiated in novel scenes. We then describe how RAPID constructs and verifies these programs from a single human demonstration in Sec. IV.

A. Primitives as Local Trajectory-Optimization Programs

A primitive is a reusable behavior for realizing an object-level motion effect, such as pushing for a displacement or flipping for an angle. We represent a primitive as a local

TABLE I: Relational program for flipping an object against a wall.

Component	Content
Semantic goal G_i	Flip the target against the vertical fixture for a specific angle.
Relational contract C_i	Semantic roles $\mathbf{r}_i$: target, support plane, fixture, and pusher. Relations Γ_i : the target rests on the support plane, and the fixture exposes a reachable vertical face. Effect parameter $\mathbf{q}_i$: flipping angle.
Success predicate Ψ_i	Check that the achieved rotation matches the flipping angle and that the target remains supported, near the fixture, and settled.
Geometry resolver ρ_i	At each rollout state, derive the support plane, fixture face, pivot axis, target faces, dimensions, and clearances.
Final cost J_i^f	Penalize terminal rotation error relative to the flipping angle, support and fixture gaps, and terminal velocity.
Progress cost J_i^p	Encourage rotation and pusher proximity while discouraging backtracking and irregular motion.

relational trajectory-optimization program

$$P_i = (G_i, C_i, \Psi_i, \rho_i, J_i) \in \mathcal{P}.$$

Here, G_i is a semantic, object-centric description of the intended effect; C_i is the relational interface; Ψ_i is the verifiable success condition; ρ_i is the geometry resolver; and J_i is the optimization cost. Together, these components specify how the primitive is instantiated, optimized, and verified in a specific scene. A primitive is therefore neither a fixed trajectory nor a black-box policy. As a concrete example, Table I illustrates each component for flipping an object against a wall. We now explain the components in detail.

Relational interface. We write the primitive interface as

$$C_i = (\mathbf{r}_i, \Gamma_i, \mathbf{q}_i).$$

The declarations $\mathbf{r}_i$ and $\mathbf{q}_i$ specify the semantic scene roles and typed effect-parameter slots required by P_i , while Γ_i specifies the relations among the roles. An invocation supplies runtime arguments (b_i, g_i) satisfying

$$b_i \in \text{Bind}_E(\mathbf{r}_i, \Gamma_i), \quad g_i \in \text{Val}(\mathbf{q}_i).$$

Here, $\text{Bind}_E(\mathbf{r}_i, \Gamma_i)$ denotes the type-compatible assignments of scene entities from the environment E to $\mathbf{r}_i$ that satisfy Γ_i , and $\text{Val}(\mathbf{q}_i)$ denotes the values matching the types declared by $\mathbf{q}_i$. Thus, b_i instantiates the role declarations and g_i instantiates the effect-parameter declarations. In the flipping example (Table I), b_i binds the target, support plane, fixture, and pusher to scene entities, while g_i supplies the requested angle. Scene-dependent geometry remains internal to the primitive and is resolved by ρ_i at execution time.

Geometry resolver. Let $\mathcal{X}(E)$ be the state space of environment E . At each state $x_t \in \mathcal{X}(E)$ along a rollout, the geometry resolver computes

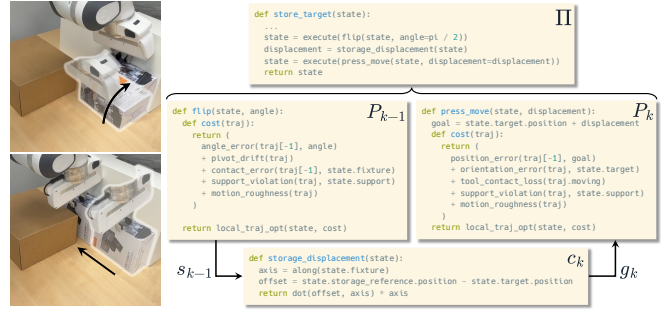
$$\phi_{i,t} = \rho_i(E, x_t, b_i), \quad t = 0, \dots, T,$$

where b_i assigns the primitive’s semantic roles to concrete entities in E , and $\phi_{i,t}$ collects the corresponding geometric features at time t .

Success predicate. For a terminal state $x_T \in \mathcal{X}(E)$, the predicate $\Psi_i(E, x_T; g_i, \phi_{i,T}) \in \{0, 1\}$ provides an executable interpretation of G_i . It evaluates the achieved object relations and requested effect, yielding an unambiguous signal for verification.

Optimization cost. For a simulated trajectory $\tau = (x_0, \dots, x_T)$ in environment E , where $x_t \in \mathcal{X}(E)$, the primitive cost is

$$J_i(E, \tau; g_i, \phi_{i,0:T}) = J_i^f(E, x_T; g_i, \phi_{i,T}) + \lambda_i J_i^p(E, \tau; g_i, \phi_{i,0:T}).$$


Fig. 3: Program

Here, J_i^f is the final cost, which measures how far x_T is from the effect encoded by G_i ; J_i^p is the progress cost, which may depend on the full trajectory and guides the optimizer through contact-rich interaction heuristics; and $\lambda_i \geq 0$ is a constant. The coding agent proposes the heuristics by reasoning about the visual human demonstration.

During primitive construction, the agent also proposes an initialization heuristic based on the resolved geometry and requested effect. The heuristic provides a plausible interaction trajectory from which to begin the search. Let z parameterize a candidate robot motion, let $\tau(z) = \text{Rollout}_E(x_0, z)$ denote its simulated state trajectory from x_0 in environment E , and let $\phi_{i,0:T}(z)$ be the feature sequence resolved along that trajectory. The executable trajectory is obtained as

$$z_i^* = \arg \min_z J_i(E, \tau(z); g_i, \phi_{i,0:T}(z)).$$

We solve this trajectory-optimization problem with the Cross-Entropy Method (CEM) [19], a sampling-based optimizer. The primitive thereby adapts its motion to the object pose, size, geometry, physical properties, and requested effect.

B. Strategies as Primitive Composition by Constraints

Suppose we have a library of primitives $\mathcal{P}$, where each $P_i \in \mathcal{P}$ realizes an object-level motion effect. A strategy Π is a reusable task-level program that composes these primitives to accomplish an overall task. It specifies the sequence of primitives to invoke, how task roles map to each primitive, and how the requested effect of each invocation is determined:

$$\Pi = (G_\Pi, C_\Pi, \Psi_\Pi, \langle (P_{i_k}, m_k, c_k) \rangle_{k=1}^K),$$

where G_Π describes the overall task goal and Ψ_Π evaluates its final outcome.

Strategy interface. A strategy exposes semantic roles and their required relations,

$$C_\Pi = (\mathbf{r}_\Pi, \Gamma_\Pi).$$

To invoke the strategy, a role binding $b \in \text{Bind}_E(\mathbf{r}_\Pi, \Gamma_\Pi)$ assigns an entity to each strategy role while satisfying Γ_Π . Primitive effect parameters are not exposed through C_Π ; instead, they are derived inside the strategy during execution.

Primitive Composition by Constraints. A strategy consists of K phases, each represented by a tuple (P_{i_k}, m_k, c_k) , where $P_{i_k} \in \mathcal{P}$ is a primitive, m_k is a role map, and c_k is a connecting constraint. For the invoked primitive P_{i_k} , m_k produces a role binding b_k that fills the role declarations $\mathbf{r}_{i_k}$, while c_k produces an effect argument g_k that fills the parameter declarations $\mathbf{q}_{i_k}$.

The connecting constraint declaratively specifies how phase k depends on the outcome of phase $k-1$ and is evaluated just in time at the beginning of phase k :

$$\begin{aligned} b_k &= m_k(b) \in \text{Bind}_E(\mathbf{r}_{i_k}, \Gamma_{i_k}), \\ g_k &= c_k(E, s_{k-1}, b) \in \text{Val}(\mathbf{q}_{i_k}), \end{aligned}$$

where $s_{k-1} \in \mathcal{X}(E)$ denotes the environment state after the execution of phase $k-1$. The phase is then executed as

$$s_k = \text{Exec}(P_{i_k}; E, s_{k-1}, g_k, b_k).$$

Here, Exec solves the local trajectory-optimization problem of P_{i_k} and executes the optimized trajectory.

For example, consider a phase that establishes target–fixture contact using a pushing primitive. The role map m_k maps the strategy’s target, support, and tool roles to the corresponding roles exposed by the primitive, producing the primitive binding b_k . The connecting constraint c_k then computes the required displacement from the current target and fixture poses and expresses it in the primitive’s relational input frame, producing the effect argument g_k .

C. Strategy Instantiation in Novel Scenes

To deploy a strategy in a novel scene, we only need to provide a semantic binding. Given a novel scene E^* and instruction l^* , RAPID uses a coding agent to rapidly generate a semantic binding $b^* \in \text{Bind}_{E^*}(\mathbf{r}_\Pi, \Gamma_\Pi)$ that assigns scene entities to the semantic roles exposed by the strategy interface. Since this step only associates the exposed semantic roles with scene entities, it is lightweight and incurs little deployment overhead. At phase k , the strategy derives the primitive binding and effect argument automatically.

Throughout this process, the strategy program Π and the referenced primitives remain frozen. Generalization therefore arises from the relational program representation, together with the adaptation of trajectory optimization to the geometry and physical properties of the new scene.

IV. AGENTIC PROGRAMMING FROM DEMONSTRATION

In Sec. III, we introduced a reusable and generalizable representation of relational manipulation programs that can be efficiently instantiated in novel scenes. We now describe how RAPID employs a coding agent to construct and iteratively refine such programs from a single visual human demonstration. Specifically, RAPID closes the agentic loop with the single demonstration by automatically inferring the task specification, generating programs the manipulation primitives referred by the strategy, and reconstructing the interactive simulation environments used for verification.

A. Reconstructing Interactive Environments

Interactive environment. To verify a manipulation program, RAPID reconstructs an interactive simulation environment from the initial RGB-D frame of the demonstration. We first use a vision-language model (VLM) to identify the objects in the scene, assign them with semantic labels, and estimate their physical properties. Then, for each semantic name, we utilize SAM 3 [20] to get a segmentation mask. The RGB-D frame

and the mask are then fed into SAM 3D [21] to reconstruct a complete mesh. Finally, we use FoundationPose [22] to register each mesh to the RGB-D frame, and assemble the reconstructed objects and robot model in MuJoCo [23]. The resulting environment preserves the scene entities, geometry, and spatial relations needed to execute the candidate program and evaluate its success predicates.

Feasible scene variants. Verifying a program only in the reconstructed environment from a single demonstration may leave accidental assumptions about the demonstration undetected. To improve reusability and generalizability, RAPID uses a task-agnostic variation sampler to generate scene variants. This sampler will randomize object appearances, sizes, poses, as well as physical parameters such as mass, friction, and elasticity. However, the randomly sampled scenes might not be feasible for a certain primitive or strategy. Therefore, we will let the coding agent to write a filter to leave out those infeasible variants based on its understanding of the primitive or the strategy. A variant is accepted only if it preserves the declared roles and relations, retains the task meaning, and remains physically feasible.

B. Primitive Construction

To construct the primitives, we first need to decompose the demonstration of the overall tasks into a sequence of interaction phases. Given the full demonstration D and its natural language description l , RAPID calls a subagent to partition D into an ordered sequence of interaction segments $(D_1, \dots, D_K)$ and assigns a local description l_i to each D_i . For each (D_i, l_i) , if the coding agent recognizes that the behavior is covered by an existing primitive in the primitive library $\mathcal{P}$, we can simply retrieve this primitive. Otherwise, the coding agent will infer a primitive specification consisting of a semantic goal G_i and a success predicate Ψ_i from (D_i, l_i) . It further reasons about the contact interactions that produces the demonstrated effect, the relational structure that should remain invariant across executions, and the quantities that might vary between invocations. For example, for the flipping primitive in Table I, the demonstrated rotation supplies the flipping-angle argument, while the required relations among the target, support, fixture, and pusher define the reusable interaction structure.

Given the inferred primitive specification and the analysis about physical interaction structure, the coding agent will generate the relational primitive contract C_i , geometry resolver ρ_i , and optimization cost J_i introduced in Sec. III-A. RAPID then iteratively verifies and revises the resulting primitive in the reconstructed interactive environment $\hat{E}_i$ as well as in automatically generated scene variants. The complete process is illustrated in Algorithm 1: RAPID reconstructs the segment-level environment $\hat{E}_i$, proposes a primitive program P_i , generates feasible scene variants, and revises the program until the success predicate Ψ_i is satisfied across the verification set $\mathcal{V}_i$. Finally, the primitive is added to the library $\mathcal{P}$.

C. Strategy Composition

After constructing the required primitives, RAPID composes them into a task-level strategy. Given the full demonstration D and instruction l , the coding agent infers a strategy specification consisting of the overall task goal G_Π and final success predicate Ψ_Π . The sequence of the interaction segments $(D_1, \dots, D_K)$ determines the ordering of the primitive composition. The agent then constructs the strategy interface C_Π and, for each phase k , selects a verified primitive P_k , defines the role map m_k , and specifies the connecting constraint c_k . Together, these components form an initial version of the strategy Π .

RAPID then iteratively verifies and revises the strategy in the reconstructed interactive environment $\hat{E}_0$ and its feasible scene variants. As illustrated by Algorithm 1, during each execution, RAPID binds the semantic roles, invokes the selected primitives sequentially, evaluates each connecting constraint just in time, and checks the final outcome with Ψ_Π . The failure signals are returned to the coding agent, and the coding agent will revise the strategy composition accordingly while keeping the verified primitives fixed, until Ψ_Π is satisfied across $\mathcal{V}_\Pi$.

V. EXPERIMENTS

In this section, we answer the following questions: (1) Can RAPID successfully close the agentic coding loop from a single visual human demonstration to construct reusable programs for nonprehensile manipulation? (2) Does our relational program representation improve the generalization of the constructed manipulation program? (3) How well does RAPID perform in a real-world robotic system?

A. Experimental Setup for Nonprehensile Manipulation

Tasks. In this section, we evaluate RAPID on 8 nonprehensile manipulation tasks. For each task, we provide a single visual human demonstration together with a language description. In all tasks, the objects are initially not graspable by the parallel gripper. The robot must therefore execute a sequence of nonprehensile manipulation primitives, such as pushing, flipping, pivoting, and toppling, to reach the target configuration. Importantly, the primitives are *not* provided to the coding agent a priori. Figure 4 illustrates the eight tasks: **T1.** Push the object toward the corridor, switch the contact face, and push it again to store it inside the corridor.

T2. Push the object toward the fixture, flip it to expose an edge, and grasp it.

T3. Move the object away from the obstacles, push it toward the fixture, flip it to expose an edge, and grasp it.

T4. Push the object toward the fixture, flip it, reposition it to expose an edge, and grasp it.

T5. Push the object toward the fixture, flip it to align with the opening, and move it into the opening for storage.

T6. Move the object to the side of the platform to expose an edge, and grasp it.

T7. Pivot the box to align it with the corridor, push it toward the corridor, and then move it to store it inside the corridor.

T8. Topple the object to expose an edge, then grasp it.

Algorithm 1: RAPID

Input: Human demonstration D , instruction l
Output: Verified primitives $\mathcal{P}$ and strategy Π
 $\{(D_i, l_i)\}_{i=1}^K \leftarrow \text{SEGMENT}(D, l)$
 $\mathcal{P} \leftarrow \emptyset$
// Primitive construction
for $i = 1, \dots, K$ **do**
 $\hat{E}_i \leftarrow \text{RECONSTRUCT}(D_i)$
 $(G_i, \Psi_i) \leftarrow \text{INFEROBJECTIVE}(D_i, l_i)$
 $g_i \leftarrow \text{INFEREFFECT}(D_i, G_i)$
 $P_i \leftarrow \text{PROPOSEPRIMITIVE}(D_i, G_i, \Psi_i, \hat{E}_i)$
 $\mathcal{V}_i \leftarrow \{\hat{E}_i\} \cup \text{VARIANTS}(\hat{E}_i, P_i)$
 $P_i \leftarrow \text{VERIFYREVISE}(P_i, \mathcal{V}_i, (g_i, l_i))$
 $\mathcal{P} \leftarrow \mathcal{P} \cup \{\text{FREEZE}(P_i)\}$
// Strategy composition
 $\hat{E}_0 \leftarrow \text{RECONSTRUCT}(D)$
 $(G_\Pi, \Psi_\Pi) \leftarrow \text{INFEROBJECTIVE}(D, l)$
 $\Pi \leftarrow \text{COMPOSE}(D, \mathcal{P}, G_\Pi, \Psi_\Pi, \hat{E}_0)$
 $\mathcal{V}_\Pi \leftarrow \{\hat{E}_0\} \cup \text{VARIANTS}(\hat{E}_0, \Pi)$
 $\Pi \leftarrow \text{VERIFYREVISE}(\Pi, \mathcal{V}_\Pi, l)$
return $\mathcal{P}, \text{FREEZE}(\Pi)$
// Shared verification and revision
function $\text{VERIFYREVISE}(Q, \mathcal{V}, a)$
 repeat
 $\mathcal{R} \leftarrow \text{EXECUTEEVALUATE}(Q, \mathcal{V}, a)$
 if $\neg \text{ALLSUCCESSFUL}(\mathcal{R})$ **then**
 $Q \leftarrow \text{DIAGNOSEREVISE}(Q, \mathcal{R})$
 until $\text{ALLSUCCESSFUL}(\mathcal{R})$
 return Q

Benchmark. We construct a simulation benchmark in MuJoCo [23] using a Franka Research 3 robot arm equipped with a compliant operational space controller. Our primary evaluation metric is *generalization*. For each of the 8 tasks, we construct 50 novel test scenes that differ from the demonstration scene in object pose, appearance, size, shape, and material. We also introduce irrelevant objects to make the test scenes visually distracting. We evaluate the success rate of each method on each task, repeat each experiment for 3 trials, and report the mean and standard deviation.

Baselines. We compare RAPID with four code-as-policy variants. (1) *Code as Policies (CaP)* [9]: for each novel test scene, it generates a program given the demonstration, language description, and observation, without simulation feedback during code generation. (2) *CaP + RPR*: we augment CaP with our relational program representation (RPR) and the cross-entropy method (CEM) optimizer, allowing motions to be expressed relative to scene entities. (3) *CaP-Agent0* [6]: given a demonstration scene, it generates and iteratively revises a program using rollout feedback, including ground-truth success signals, environment states, and visual observations. At evaluation time, it is also allowed to perform rollouts

TABLE II: Success rates on nonprehensile manipulation tasks in simulation. We report the mean and standard deviation across three trials. **Bold** denotes the best mean or results not significantly worse according to a one-sided sign test with Holm correction ($\alpha = 0.05$).

Method	Task 1	Task 2	Task 3	Task 4	Task 5	Task 6	Task 7	Task 8	Average
CaP	0.053 $\pm$ 0.009	0.000 $\pm$ 0.000	0.000 $\pm$ 0.000	0.000 $\pm$ 0.000	0.000 $\pm$ 0.000	0.000 $\pm$ 0.000	0.013 $\pm$ 0.009	0.040 $\pm$ 0.016	0.013 $\pm$ 0.003
CaP + RPR	0.040 $\pm$ 0.000	0.000 $\pm$ 0.000	0.000 $\pm$ 0.000	0.000 $\pm$ 0.000	0.000 $\pm$ 0.000	0.000 $\pm$ 0.000	0.013 $\pm$ 0.009	0.033 $\pm$ 0.025	0.011 $\pm$ 0.002
CaP-Agent0	0.353 $\pm$ 0.115	0.040 $\pm$ 0.057	0.000 $\pm$ 0.000	0.013 $\pm$ 0.019	0.173 $\pm$ 0.176	0.053 $\pm$ 0.041	0.240 $\pm$ 0.128	0.293 $\pm$ 0.242	0.146 $\pm$ 0.025
CaP-Agent0 + SV	0.440 $\pm$ 0.141	0.053 $\pm$ 0.009	0.020 $\pm$ 0.028	0.047 $\pm$ 0.066	0.113 $\pm$ 0.034	0.080 $\pm$ 0.059	0.167 $\pm$ 0.066	0.087 $\pm$ 0.096	0.126 $\pm$ 0.043
RAPID w/o SV	0.933 $\pm$ 0.009	0.833 $\pm$ 0.125	0.400 $\pm$ 0.161	0.440 $\pm$ 0.315	0.447 $\pm$ 0.256	0.340 $\pm$ 0.236	0.567 $\pm$ 0.229	0.293 $\pm$ 0.415	0.532 $\pm$ 0.071
RAPID	0.933 $\pm$ 0.057	0.847 $\pm$ 0.025	0.700 $\pm$ 0.033	0.787 $\pm$ 0.082	0.793 $\pm$ 0.009	0.587 $\pm$ 0.050	0.807 $\pm$ 0.203	0.620 $\pm$ 0.028	0.759 $\pm$ 0.024

on each test scene and revise the program. Notably, RAPID requires neither ground-truth success signals nor evaluation-time interaction. (4) *CaP-Agent0* + SV: we augment CaP-Agent0 with our scene variant (SV) generation, allowing the agent to further refine its program through rollouts on scene variants. We do not compare with ASPIRE [5] on nonprehensile tasks because ASPIRE requires pre-given motion primitives. We use Codex [1] powered by GPT-5.6 Sol with high reasoning effort as the coding agent for all methods.

B. Results and Analysis for Nonprehensile Manipulation

RAPID successfully constructs reusable programs by closing the agentic loop from a single demonstration. As shown in Table II, vanilla CaP performs poorly, largely because it lacks an environment in which generated programs can be verified and refined. Notably, its performance remains poor even when it is allowed to generate a new program for each test scene. In contrast, RAPID reconstructs a simulation environment from the demonstration, infers a success predicate from the demonstration and language instruction, and uses this predicate to verify and iteratively refine candidate programs in simulation, thereby closing the agentic coding loop. This enables RAPID to construct a reusable program for each task that generalizes across diverse test scenes. Interestingly, guiding CaP to generate programs using our relational program representation does not improve its performance. This suggests that representation alone is insufficient: without closed-loop verification, the coding agent struggles to correctly reason about object relations and therefore fails to construct meaningful relational programs.

The relational program representation enhances the generalization of the strategy program. The comparison between RAPID and the two CaP-Agent0 baselines in Table II, which do not use our relational program representation, demonstrates the importance of explicitly encoding relational structure. Even without ground-truth success signals or evaluation-time rollouts and program refinement, RAPID achieves higher success rates than CaP-Agent0. We attribute this improvement to the explicit decomposition of manipulation strategies into motion primitives, which are constructed locally and composed through relational constraints. Although CaP-Agent0 incorporates state feedback, its generated policies often use fixed world-frame offsets and predetermined timing, limiting spatial generalization and robustness to state deviations during execution. In contrast, RAPID computes geometry-dependent subgoals from the state reached after each preceding phase.

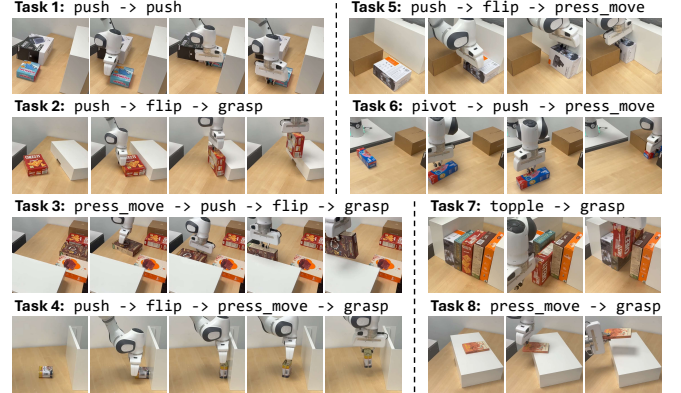


Fig. 4: Real-world execution of the solution trajectories found by RAPID.

The gap is even larger for contact-rich, forceful behaviors such as flipping and toppling, which are naturally expressed by our relational, optimization-based motion primitives.

Ablation study of scene variant generation. To evaluate the effectiveness of scene variant generation, we remove it from RAPID and report the results in Table II. Except for Tasks 1 and 2, the performance of RAPID drops substantially, while the standard deviation of the success rate increases, indicating reduced stability and robustness. Interestingly, augmenting CaP-Agent0 with scene variant generation does not improve its performance. This suggests that, without our relational program representation, CaP-Agent0 struggles to construct a generalizable program that can solve diverse generated variants without overfitting to individual scenes.

C. Experiments on LIBERO-Pro

Experimental Setup. To demonstrate the general applicability of RAPID beyond non-prehensile manipulation, we evaluate it on LIBERO-Pro [18], an extension of LIBERO [24] with stronger position (Pos.) and instruction (Task.) perturbations. LIBERO-Pro predominantly features prehensile tasks, including pick-and-place and grasp-based articulated-object manipulation. We compare against two groups of baselines: (1) vision-language-action (VLA) models, including OpenVLA [25], π_0 [26], and $\pi_{0.5}$ [27], whose results are directly taken from the LIBERO-Pro paper; and (2) code-as-policy (CaP) methods, including CaP-Agent0 [6] and ASPIRE [5]. For a fair comparison with CaP methods, which do not use demonstrations, RAPID also receives no demonstrations and is given only the task description and a single debug scene. This setting is more restrictive than ASPIRE, which uses 15 debug scenes per task. Moreover, while CaP baselines are provided with predefined motion primitives such as grasping, placing, and transporting, RAPID must discover and implement these

TABLE III: Experiment Results on LIBERO-Pro

Method	libero-object		libero-goal		libero-spatial	
	Pos. (Avg.)	Task. (Avg.)	Pos. (Avg.)	Task. (Avg.)	Pos. (Avg.)	Task. (Avg.)
OpenVLA	0	0	0	0	0	0
π_0	0	0	0	0	0	0
$\pi_{0.5}$	0.17	0.01	0.38	0	0.20	0.01
CaP-Agent0	0.22	0.18	0.26	0.17	0.12	0.14
ASPIRE	0.98	0.95	0.81	0.45	0.51	0.60
RAPID	1	0.95	0.67	0.53	0.83	0.81

primitives solely through exploration and iterative refinement in the debug scene.

Results. Table III reports the results on LIBERO-Pro. RAPID achieves the highest success rate on five of the six task settings, demonstrating the effectiveness of our relational program representation. On LIBERO-Goal (Pos.), RAPID ranks second behind ASPIRE. This gap stems from articulated-object manipulation: primitives discovered by RAPID from only a single debug scene, such as drawer opening, are less robust to spatial variation, whereas ASPIRE is provided with an *interpolate segment* motion primitive that simplifies such interactions. Overall, these results demonstrate the generality of our framework beyond non-prehensile manipulation.

D. Real-World Experiments for Nonprehensile Manipulation

To demonstrate that RAPID can operate effectively on a real robotic system, we conduct real-world experiments on the eight nonprehensile manipulation tasks shown in Figure 4. As illustrated in Figure 5, our setup consists of a Franka Research 3 robot arm equipped with a parallel gripper and an Intel RealSense L515 camera for RGB-D observations. We compare RAPID against CaP-Agent0 [6]. For each task, we evaluate 10 test scenes and report the success rate.

Results. As shown in Figure 6, with our relational program representation and compliant operational-space controller, contact-rich nonprehensile behaviors optimized in simulation can be transferred directly to the real robot and executed robustly. In contrast, CaP-Agent0 struggles to generate reliable trajectories using purely procedural programs. Moreover, leveraging the VLM’s prior knowledge to estimate physical parameters such as mass, friction, and restitution, together with SAM3D’s strong 3D reconstruction capability, helps keep the real-to-sim and sim-to-real gaps manageable.

Failure Analysis.

VI. RELATED WORK

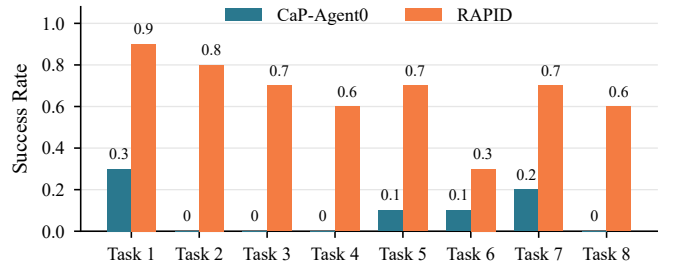
Agentic Programming for Robotics. Learning from Few Demonstrations. Nonprehensile Manipulation.

VII. CONCLUSION

APPENDIX

REFERENCES

- [1] OpenAI, “Codex,” 2025. Website: <https://openai.com/codex/>.
- [2] Anthropic, “Claude Code,” 2025. Website: <https://claude.com/product/claude-code>.
- [3] K. Chen, S. Xie, L. Fu, J. Yu, W. Pacini, S. Bajamahal, H. Kim, J. Drake, D. Kim, H. Xue, *et al.*, “GaP: A Graph-as-Policy Multi-Agent Self-Learning Harness For Variational Automation Tasks,” *arXiv:2607.05369*, 2026.

**Fig. 5:** Experiment setup.**Fig. 6:** Success rates across eight real-world manipulation tasks.

- [4] W. Xiao, J. Xie, T. Zhang, H. Lin, L. Fu, H. Xue, J. Lu, Y. Yang, C. Dai, Z. Wang, *et al.*, “ENPIRE: Agentic Robot Policy Self-Improvement in the Real World,” *arXiv:2606.19980*, 2026.
- [5] R. Lu, Y. Wu, E. Kou, L. Fu, W. Xiao, A. Mandlekar, Y. Xu, G. Shi, K. Goldberg, A. Chen, *et al.*, “ASPIRE: Agentic Skills Discovery for Robotics,” *arXiv:2607.00272*, 2026.
- [6] L. Fu, J. Yu, K. El-Refai, E. Kou, H. Xue, H. Huang, W. Xiao, G. Wang, D. Niu, F.-F. Li, *et al.*, “CaP-X: A framework for benchmarking and improving coding agents for robot manipulation,” *arXiv:2603.22435*, 2026.
- [7] S. Berman, M. Ilie, J. Deng, and D. Freeman, “Claude Plays Robotics,” 2026. Research Blog: <https://www.anthropic.com/research/claude-plays-robotics>.
- [8] Noematrix Team, “RoboRSI: Stable, Efficient, and Reusable Robot Self-Evolution in Complex Real-World Environments,” 2026. Research Blog: <https://lab.noematrix.ai/blog/2-robors/>.
- [9] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, “Code as Policies: Language Model Programs for Embodied Control,” in *ICRA*, 2023.
- [10] A. Ng, “Three Key Loops for Building Great Software,” 2026. Research Blog: <https://www.deeplearning.ai/the-batch/three-key-loops-for-building-great-software>.
- [11] D. Huang, J. M. Zhang, M. Luck, Q. Bu, Y. Qing, and H. Cui, “AgentCoder: Multi-Agent-Based Code Generation with Iterative Testing and Optimisation,” *arXiv:2312.13010*, 2023.
- [12] C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez, “Integrated Task and Motion Planning,” *Annual review of control, robotics, and autonomous systems*, vol. 4, pp. 265–293, 2021.
- [13] M. T. Mason, *Mechanics of Robotic Manipulation*. MIT press, 2001.
- [14] A. Holladay, R. Paolini, and M. T. Mason, “A General Framework for Open-Loop Pivoting,” in *ICRA*, 2015.
- [15] X. Cheng, E. Huang, Y. Hou, and M. T. Mason, “Contact Mode Guided Sampling-Based Planning for Quasistatic Dexterous Manipulation in 2D,” in *ICRA*, 2021.
- [16] L. U. Odhner, R. R. Ma, and A. M. Dollar, “Open-Loop Precision

Grasping with Underactuated Hands Inspired by a Human Manipulation Strategy,” *T-ASE*, vol. 10, no. 3, pp. 625–633, 2013.

- [17] X. Cheng, E. Huang, Y. Hou, and M. T. Mason, “Contact Mode Guided Motion Planning for Quasidynamic Dexterous Manipulation in 3D,” in *ICRA*, 2022.
- [18] X. Zhou, Y. Xu, G. Tie, Y. Chen, G. Zhang, D. Chu, P. Zhou, and L. Sun, “LIBERO-PRO: Towards Robust and Fair Evaluation of Vision-Language-Action Models beyond Memorization,” *arXiv:2510.03827*, 2025.
- [19] P.-T. de Boer, D. P. Kroese, S. Mannor, and R. Y. Rubinstein, “A Tutorial on the Cross-Entropy Method,” *Annals of Operations Research*, vol. 134, no. 1, pp. 19–67, 2005.
- [20] N. Carion, L. Gustafson, Y.-T. Hu, S. Debnath, R. Hu, D. Suris Coll-Vinent, C. Ryali, K. V. Alwala, H. Khedr, A. Huang, *et al.*, “SAM 3: Segment Anything with Concepts,” in *ICLR*, 2026.
- [21] X. Chen, F.-J. Chu, P. Gleize, K. J. Liang, A. Sax, H. Tang, W. Wang, M. Guo, T. Hardin, X. Li, *et al.*, “SAM 3D: 3Dfy Anything in Images,” in *CVPR*, 2026.
- [22] B. Wen, W. Yang, J. Kautz, and S. Birchfield, “FoundationPose: Unified 6D Pose Estimation and Tracking of Novel Objects,” in *CVPR*, 2024.
- [23] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A Physics Engine for Model-Based Control,” in *IROS*, 2012.
- [24] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone, “LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning,” in *NeurIPS*, 2023.
- [25] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, *et al.*, “OpenVLA: An Open-Source Vision-Language-Action Model,” *arXiv:2406.09246*, 2024.
- [26] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, *et al.*, “ π_0 : A Vision-Language-Action Flow Model for General Robot Control,” *arXiv:2410.24164*, 2024.
- [27] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, *et al.*, “ $\pi_{0.5}$: A Vision-Language-Action Model with Open-World Generalization,” *arXiv:2504.16054*, 2025.